

LAPORAN PRAKTIKUM ASD



Muhammad Saum Saifulloh

254107060155

Sistem Informasi Bisnis / 1E

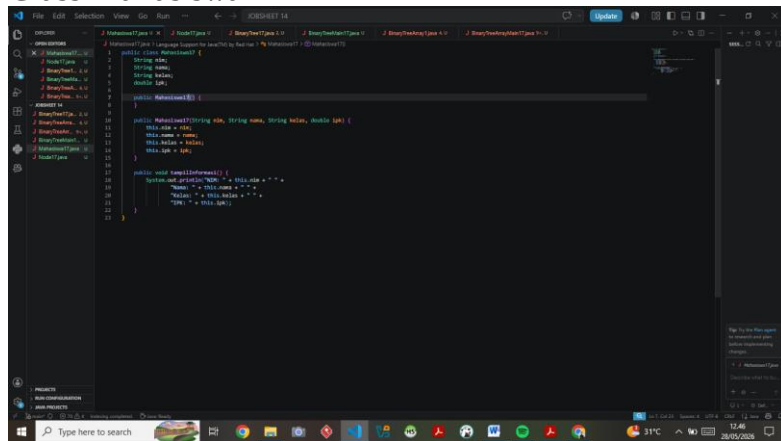
JOBSHEET 14 Tree

Tujuan Praktikum

Setelah melakukan praktikum ini, mahasiswa mampu:

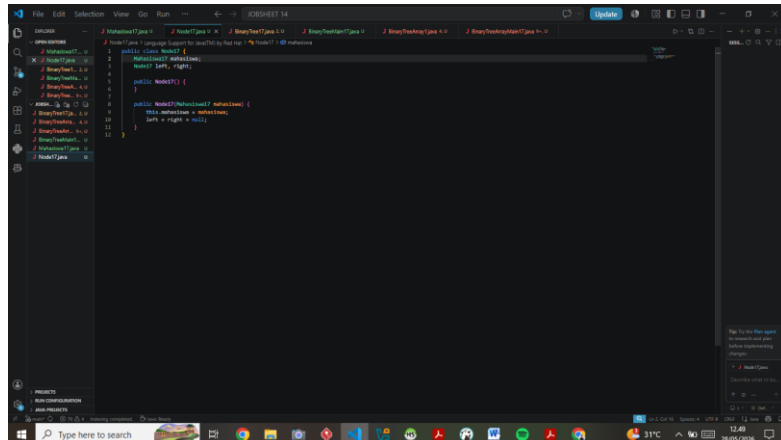
1. memahami model Tree khususnya Binary Tree
2. membuat dan mendeklarasikan struktur algoritma Binary Tree.
3. menerapkan dan mengimplementasikan algoritma Binary Tree dalam kasus Binary Search Tree

14.2 Kegiatan Praktikum 1 — Implementasi Binary Search Tree menggunakan Linked List Class Mahasiswa



```
1 public class Mahasiswa {
2     String nama;
3     String nim;
4     String kelas;
5     double ipk;
6
7     public Mahasiswa() {
8     }
9
10    public Mahasiswa(String nama, String nim, String kelas, double ipk) {
11        this.nama = nama;
12        this.nim = nim;
13        this.kelas = kelas;
14        this.ipk = ipk;
15    }
16
17    public void tampilInformasi() {
18        System.out.println("Nama = " + this.nama + " ");
19        System.out.println("NIM = " + this.nim + " ");
20        System.out.println("Kelas = " + this.kelas + " ");
21        System.out.println("IPK = " + this.ipk);
22    }
23 }
```

Class Node



```
1 public class Node {
2     Mahasiswa mahasiswa;
3     Node left, right;
4
5     public Node() {
6     }
7
8     public Node(Mahasiswa mahasiswa) {
9         this.mahasiswa = mahasiswa;
10        left = right = null;
11    }
12 }
```

Class BinaryTree

```
// Main class: Main.java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Create root node
        Node root = new Node(1);
        // Insert nodes
        root = insert(root, 2);
        root = insert(root, 3);
        root = insert(root, 4);
        root = insert(root, 5);
        root = insert(root, 6);
        root = insert(root, 7);
        // Traverse
        System.out.println("Pre-order traversal:");
        preOrder(root);
        System.out.println("In-order traversal:");
        inOrder(root);
        System.out.println("Post-order traversal:");
        postOrder(root);
        // Delete
        root = delete(root, 3);
        System.out.println("Tree after deleting 3:");
        preOrder(root);
    }
}

// Node class
class Node {
    int data;
    Node left;
    Node right;
    Node(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}

// BinaryTree class
class BinaryTree {
    // Insert
    public Node insert(Node root, int data) {
        if (root == null)
            return new Node(data);
        if (data < root.data)
            root.left = insert(root.left, data);
        else if (data > root.data)
            root.right = insert(root.right, data);
        return root;
    }

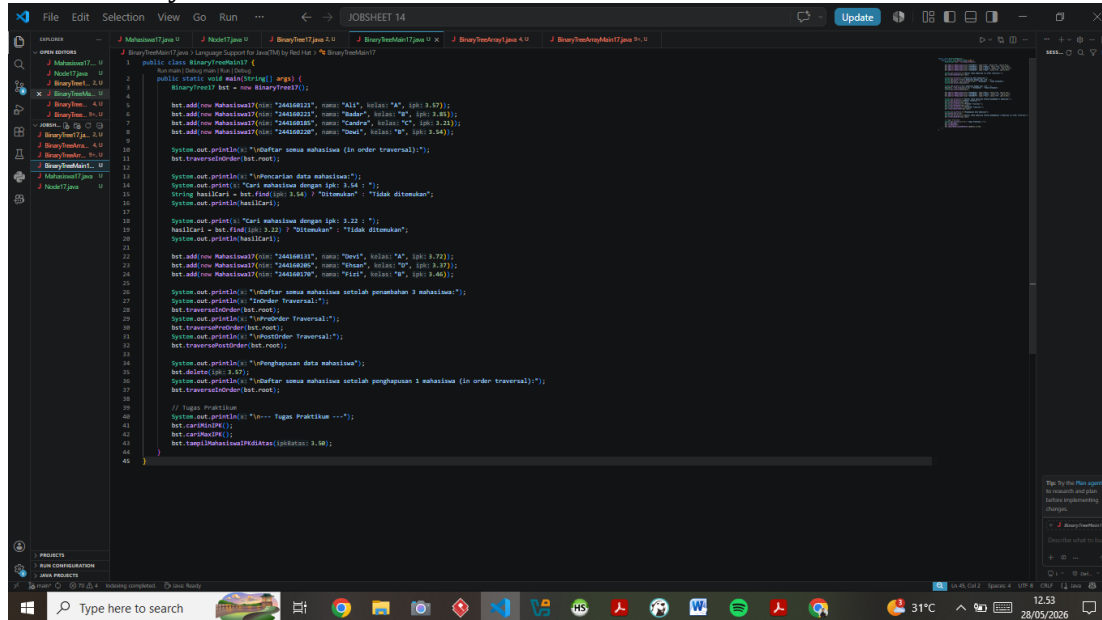
    // Pre-order traversal
    public void preOrder(Node root) {
        if (root == null)
            return;
        System.out.print(root.data + " ");
        preOrder(root.left);
        preOrder(root.right);
    }

    // In-order traversal
    public void inOrder(Node root) {
        if (root == null)
            return;
        inOrder(root.left);
        System.out.print(root.data + " ");
        inOrder(root.right);
    }

    // Post-order traversal
    public void postOrder(Node root) {
        if (root == null)
            return;
        postOrder(root.left);
        postOrder(root.right);
        System.out.print(root.data + " ");
    }

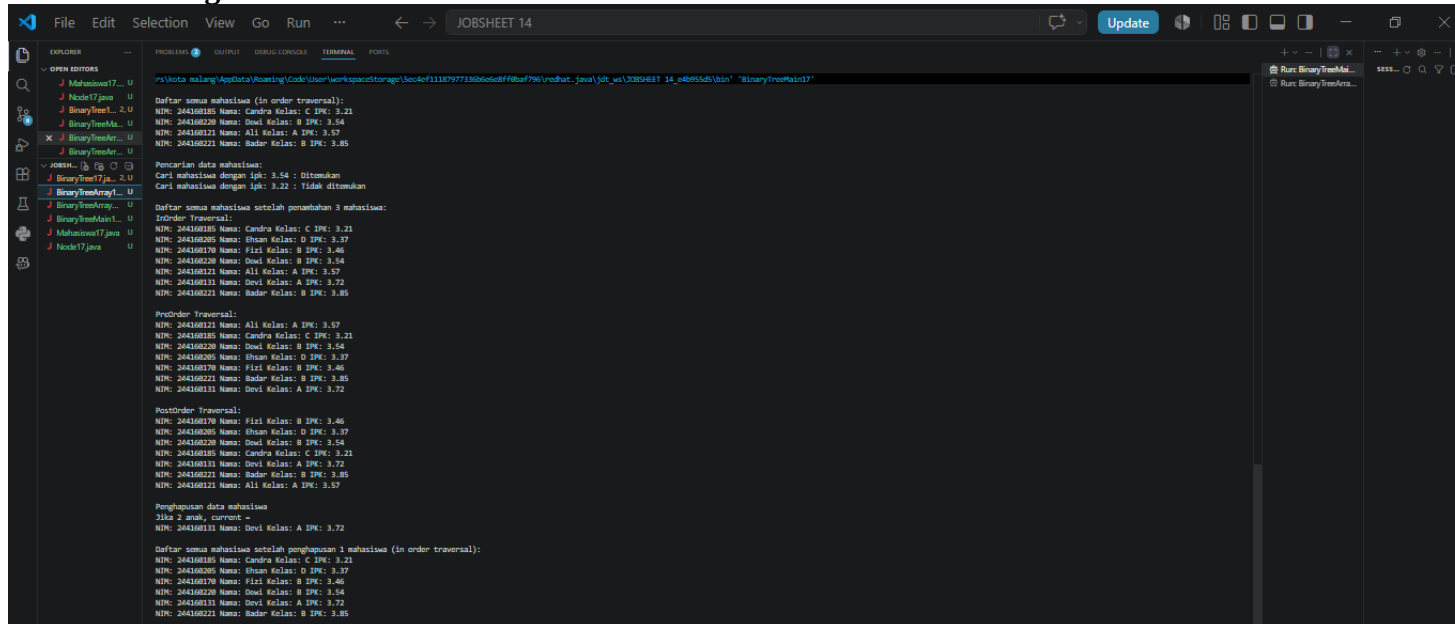
    // Delete
    public Node delete(Node root, int data) {
        if (root == null)
            return null;
        if (data < root.data)
            root.left = delete(root.left, data);
        else if (data > root.data)
            root.right = delete(root.right, data);
        else {
            // Node to be deleted
            if (root.left == null && root.right == null)
                return null;
            else if (root.left == null)
                return root.right;
            else if (root.right == null)
                return root.left;
            // Both children exist
            Node temp = new Node(root.data);
            root.left = delete(root.left, root.data);
            root.right = delete(root.right, root.data);
            return temp;
        }
    }
}
```

Class BinaryTreeMain



```
1 public class BinaryTreeMain {
2     public static void main(String[] args) {
3         BinaryTree<Mahasiswa> bt = new BinaryTree<>();
4
5         bt.add(new Mahasiswa(1, "244108121", "Nama: 'Ali', kelas: 'A', ipk: 3.87));
6         bt.add(new Mahasiswa(2, "244108221", "Nama: 'Bader', kelas: 'B', ipk: 3.54));
7         bt.add(new Mahasiswa(3, "244108220", "Nama: 'Dewi', kelas: 'C', ipk: 3.72));
8         bt.add(new Mahasiswa(4, "244108220", "Nama: 'Dewi', kelas: 'B', ipk: 3.54));
9
10        System.out.println("Daftar semua mahasiswa (in order traversal):");
11        bt.traversalInOrder(bt.root);
12
13        System.out.println("Pencarian data mahasiswa:");
14        System.out.println("Cari mahasiswa dengan ipk: 3.54:");
15        String hasilCari = bt.find(ipk: 3.54) ? "Ditemukan" : "Tidak ditemukan";
16        System.out.println(hasilCari);
17
18        System.out.println("Cari mahasiswa dengan ipk: 3.22:");
19        hasilCari = bt.find(ipk: 3.22) ? "Ditemukan" : "Tidak ditemukan";
20        System.out.println(hasilCari);
21
22        bt.add(new Mahasiswa(5, "244108131", "Nama: 'Dewi', kelas: 'A', ipk: 3.72));
23        bt.add(new Mahasiswa(6, "244108205", "Nama: 'Ethan', kelas: 'D', ipk: 3.37));
24        bt.add(new Mahasiswa(7, "244108170", "Nama: 'Fizi', kelas: 'B', ipk: 3.46));
25
26        System.out.println("Daftar semua mahasiswa setelah penambahan 3 mahasiswa:");
27        System.out.println("InOrder Traversal:");
28        bt.traversalInOrder(bt.root);
29        System.out.println("PreOrder Traversal:");
30        bt.traversalPreOrder(bt.root);
31        System.out.println("PostOrder Traversal:");
32        bt.traversalPostOrder(bt.root);
33
34        System.out.println("Penghapusan data mahasiswa:");
35        bt.delete(ipk: 3.57);
36        System.out.println("Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):");
37        bt.traversalInOrder(bt.root);
38
39        // Tugas Praktikum
40        System.out.println("===== Tugas Praktikum =====");
41        bt.cariMaxIpk();
42        bt.cariMaxIpk();
43        bt.tampilMahasiswaPerKelas(ipk: 3.54);
44    }
45 }
```

Hasil Run Program



```
Daftar semua mahasiswa (in order traversal):
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87
NIM: 244108220 Nama: 'Dewi' kelas: 'B' IPK: 3.54
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72
NIM: 244108221 Nama: 'Bader' kelas: 'B' IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87
NIM: 244108205 Nama: 'Ethan' kelas: 'D' IPK: 3.37
NIM: 244108170 Nama: 'Fizi' kelas: 'B' IPK: 3.46
NIM: 244108220 Nama: 'Dewi' kelas: 'B' IPK: 3.54
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72
NIM: 244108221 Nama: 'Bader' kelas: 'B' IPK: 3.85

PreOrder Traversal:
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87
NIM: 244108205 Nama: 'Ethan' kelas: 'D' IPK: 3.37
NIM: 244108220 Nama: 'Dewi' kelas: 'B' IPK: 3.54
NIM: 244108170 Nama: 'Fizi' kelas: 'B' IPK: 3.46
NIM: 244108221 Nama: 'Bader' kelas: 'B' IPK: 3.85
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72

PostOrder Traversal:
NIM: 244108170 Nama: 'Fizi' kelas: 'B' IPK: 3.46
NIM: 244108205 Nama: 'Ethan' kelas: 'D' IPK: 3.37
NIM: 244108220 Nama: 'Dewi' kelas: 'B' IPK: 3.54
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72
NIM: 244108221 Nama: 'Bader' kelas: 'B' IPK: 3.85
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244108121 Nama: 'Ali' kelas: 'A' IPK: 3.87
NIM: 244108205 Nama: 'Ethan' kelas: 'D' IPK: 3.37
NIM: 244108170 Nama: 'Fizi' kelas: 'B' IPK: 3.46
NIM: 244108220 Nama: 'Dewi' kelas: 'B' IPK: 3.54
NIM: 244108131 Nama: 'Dewi' kelas: 'A' IPK: 3.72
NIM: 244108221 Nama: 'Bader' kelas: 'B' IPK: 3.85
```

14.2.2 Jawaban Pertanyaan Percobaan 1

1. Mengapa dalam Binary Search Tree proses pencarian data bisa lebih efektif dibanding binary tree biasa?

Karena BST memiliki aturan terurut: semua node di subtree kiri selalu memiliki nilai lebih kecil dari node induknya, dan semua node di subtree kanan selalu lebih besar. Dengan aturan ini, setiap langkah pencarian dapat langsung membuang setengah dari sisa tree (mirip binary search). Kompleksitas pencarian rata-rata adalah $O(\log n)$, jauh lebih efisien dibanding binary tree biasa yang harus mengunjungi semua node satu per satu dengan kompleksitas $O(n)$.

2. Untuk apakah di class Node kegunaan dari atribut left dan right?

Atribut `left` adalah referensi (pointer) ke node anak sebelah kiri, yaitu node yang memiliki nilai IPK lebih kecil dari node saat ini. Atribut `right` adalah referensi ke node anak sebelah kanan, yaitu node yang memiliki nilai IPK lebih besar atau sama. Kedua atribut inilah yang membentuk struktur linked/terhubung antar node sehingga tree bisa terbentuk dan ditelusuri.

3a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?

Atribut `root` adalah referensi ke node paling atas (akar) dari tree. Seluruh operasi pada BST (add, find, traverse, delete) dimulai dari `root`. Tanpa `root`, tidak ada titik masuk untuk mengakses tree sama sekali.

b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?

Nilai `root` adalah `null`, karena di konstruktor `BinaryTree00()` diinisialisasi dengan `root = null`, yang berarti tree masih kosong dan belum ada node satu pun.

4. Ketika tree masih kosong dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Method `add()` pertama-tama memanggil `isEmpty()` untuk mengecek apakah `root == null`. Karena tree kosong, kondisi ini bernilai `true`, sehingga node baru langsung dijadikan `root`. Tidak ada perbandingan IPK yang dilakukan karena tidak ada node lain.

5. Jelaskan secara detail untuk apa baris program berikut dalam method `add()`:

```
java
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

Penjelasan baris per baris:

- `parent = current` → menyimpan posisi node saat ini sebagai parent, karena ketika `current` bergerak ke bawah dan menjadi null, kita masih butuh referensi ke node induknya untuk menempelkan node baru.
 - `if (mahasiswa.ipk < current.mahasiswa.ipk)` → membandingkan IPK mahasiswa baru dengan node saat ini. Jika lebih kecil, arah navigasi ke kiri.
- `current = current.left` → pindah ke anak kiri untuk melanjutkan pencarian posisi yang tepat.
- `if (current == null)` → jika posisi kiri sudah kosong, berarti ini adalah tempat yang tepat untuk menyisipkan node baru.
 - `parent.left = newNode` → tempelkan node baru sebagai anak kiri dari parent.
 - `return` → hentikan loop karena node sudah berhasil ditambahkan.
 - Blok `else` melakukan logika yang sama tetapi ke arah kanan (IPK lebih besar atau sama).

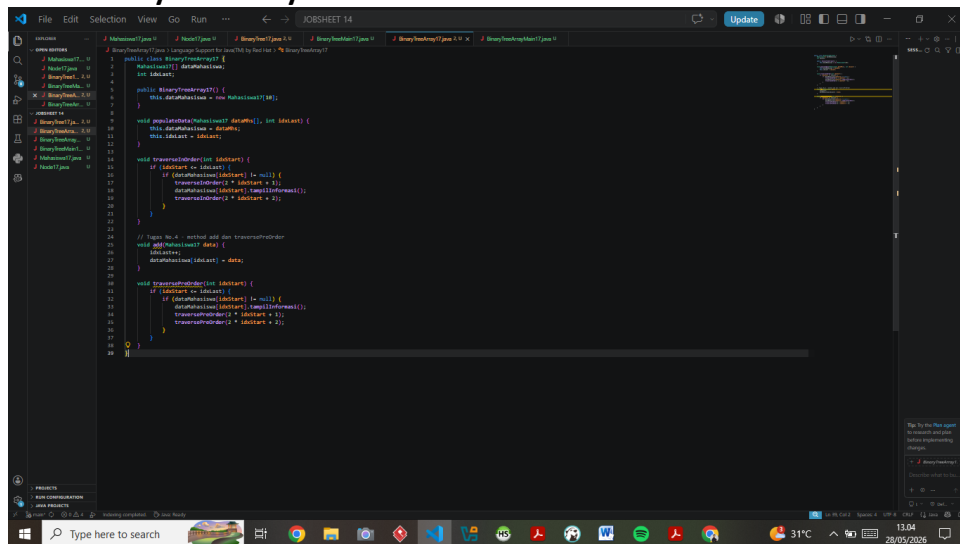
6. Jelaskan langkah-langkah pada method `delete()` saat menghapus node yang memiliki dua anak. Bagaimana `getSuccessor()` membantu?

Langkah-langkahnya:

1. Cari node yang akan dihapus (`current`) beserta parent-nya dengan menelusuri tree sesuai nilai IPK.
2. Ketika ditemukan bahwa `current.left != null` dan `current.right != null` (punya dua anak), maka dipanggil `getSuccessor(current)`.
3. `getSuccessor()` bekerja dengan: masuk ke subtree kanan dari node yang dihapus, lalu terus bergerak ke kiri sejauh mungkin untuk menemukan node dengan nilai terkecil di subtree kanan tersebut. Node ini disebut in-order successor, yaitu nilai terkecil yang masih lebih besar dari node yang dihapus.
4. Jika successor bukan anak langsung kanan dari `current`, maka `successorParent.left` diputus dan dialihkan ke `successor.right`, lalu `successor.right` diarahkan ke `del.right` agar subtree kanan tidak hilang.
5. Successor kemudian menggantikan posisi `current` di tree, dengan `successor.left` diisi oleh `current.left`.
6. Dengan cara ini, properti BST tetap terjaga karena successor adalah nilai yang "pas" menggantikan node yang dihapus.

14.3 Kegiatan Praktikum 2 — Implementasi Binary Tree dengan Array

Class `BinaryTreeArray`



```
1 package com.example.binarytreearray;
2
3 import java.util.*;
4
5 public class BinaryTreeArray {
6     private int[] data;
7     private int size;
8
9     public BinaryTreeArray() {
10         this.data = new int[100];
11     }
12
13     // Method untuk menambahkan data ke dalam array
14     public void addData(int data) {
15         this.data[size] = data;
16         size++;
17     }
18
19     // Method untuk mencari in-order successor
20     public int getSuccessor(int id) {
21         if (id < 0) return -1;
22         if (id >= size) return -1;
23         int successor = id + 1;
24         while (successor < size) {
25             if (data[successor] < data[id]) {
26                 id = successor;
27             }
28             successor++;
29         }
30         return id;
31     }
32
33     // Method untuk menghapus node
34     public void delete(int id) {
35         if (id < 0 || id >= size) return;
36         int successor = getSuccessor(id);
37         if (successor == -1) return;
38         // Menghapus node yang memiliki dua anak
39         if (id <= size - 2) {
40             // Mengalihkan pointer ke successor
41             data[id] = data[successor];
42             // Mengalihkan pointer ke successor ke kanan
43             if (successor < size - 1) {
44                 data[successor + 1] = data[successor + 2];
45             }
46         }
47         // Menghapus node yang memiliki satu anak
48         if (id < size - 1) {
49             data[id] = data[successor + 1];
50         }
51         // Menghapus node yang merupakan anak terakhir
52         if (id < size - 2) {
53             data[id] = data[successor + 2];
54         }
55         size--;
56     }
57
58     // Method untuk menampilkan data
59     public void display() {
60         for (int i = 0; i < size; i++) {
61             System.out.print(data[i] + " ");
62             if (i % 10 == 9) {
63                 System.out.println();
64             }
65         }
66     }
67 }
```

Class BinaryTreeArrayMain

[illegible]

Hasil Run Program

[illegible]

14.3.2 Jawaban Pertanyaan Percobaan 2

1. Apakah kegunaan dari atribut data dan idxLast di class BinaryTreeArray?

`dataMahasiswa (data)` adalah array yang menyimpan seluruh node-node tree secara berurutan berdasarkan level. Indeks 0 adalah root, indeks 1 dan 2 adalah anak kiri dan kanan root, dan seterusnya mengikuti formula indeks anak = $2*i+1$ (kiri) dan $2*i+2$ (kanan). `idxLast` menyimpan indeks terakhir array yang terisi data, digunakan sebagai batas atas saat traversal agar tidak mengakses elemen kosong di luar data yang ada.

2. Apakah kegunaan dari method populateData()?

Method `populateData()` digunakan untuk mengisi/memuat data ke dalam objek `BinaryTreeArray` dari luar class. Method ini menerima array mahasiswa dan nilai `idxLast` sebagai parameter, kemudian menyimpannya ke atribut class. Dengan cara ini, data tree dapat dipersiapkan di method `main()` lalu dimasukkan sekaligus ke dalam objek tree.

3. Apakah kegunaan dari method traverseInOrder()?

Method `traverseInOrder()` digunakan untuk mengunjungi dan menampilkan seluruh node tree secara in-order, yaitu dengan urutan: kunjungi subtree kiri terlebih dahulu, lalu tampilkan node saat ini, kemudian kunjungi subtree kanan. Pada BST berbasis array, hasilnya akan menampilkan data mahasiswa dari IPK terkecil ke terbesar karena sifat in-order traversal yang menghasilkan data terurut ascending.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Menggunakan formula:

- Left child = $2 * \text{indeks} + 1 = 2 * 2 + 1 = \text{indeks } 5$
- Right child = $2 * \text{indeks} + 2 = 2 * 2 + 2 = \text{indeks } 6$

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

`idxLast = 6` menyatakan bahwa data terakhir yang valid dalam array berada di indeks ke-6. Artinya tree memiliki 7 node (indeks 0 sampai 6). Nilai ini digunakan oleh method `traverseInOrder()` sebagai syarat batas (`idxStart <= idxLast`) agar rekursi berhenti dan tidak menelusuri elemen array yang bernilai `null` atau kosong di luar rentang data.

6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif?

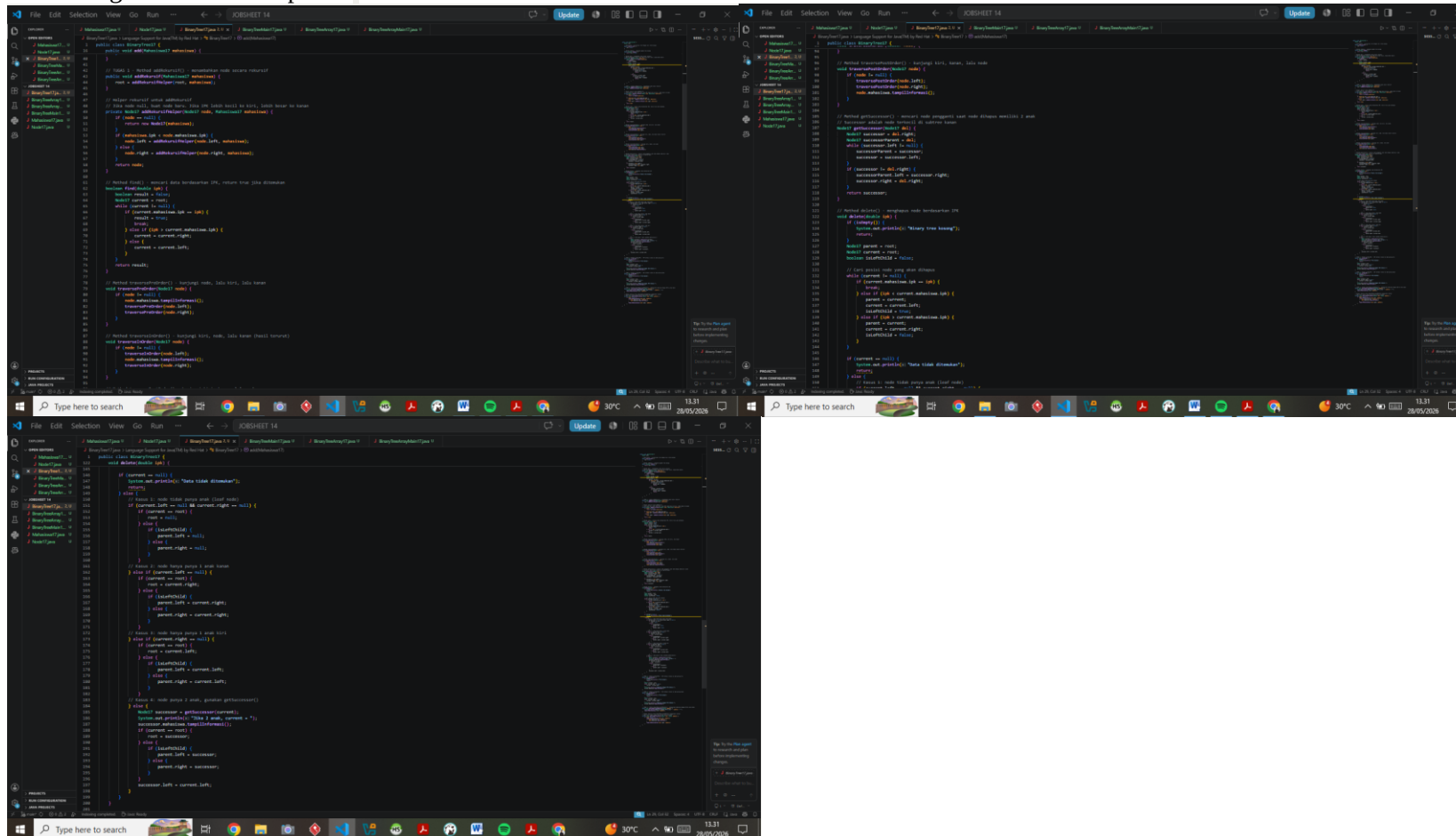
Ini adalah formula matematis standar untuk merepresentasikan binary tree dalam array. Jika sebuah node berada di indeks i , maka:

- Anak kirinya selalu berada di indeks $2*i + 1$
- Anak kanannya selalu berada di indeks $2*i + 2$
- Sebaliknya, induk dari node di indeks i berada di indeks $(i-1) / 2$

Formula ini bekerja karena array menyimpan node level per level dari kiri ke kanan (level-order), sehingga hubungan induk-anak bisa dihitung langsung secara aritmatik tanpa perlu pointer seperti pada linked list.

14.4 Tugas Praktikum

Tugas 1 — addRekursif() (sudah ada di BinaryTree00.java di atas) Method addRekursif() memanggil helper private addRekursifHelper(node, mahasiswa). Jika node null, buat node baru dan kembalikan. Jika IPK lebih kecil, rekursif ke kiri; jika lebih besar atau sama, rekursif ke kanan. Node hasil rekursi selalu dikembalikan dan disambungkan kembali ke parent.



[illegible]

The image shows a Windows 10 desktop environment. The primary focus is a code editor window titled "XORSHOOT 14" with a dark theme. The editor contains Java code for a binary tree structure. The code includes a `Node` class with `data` and `next` attributes, and a `BinaryTreeNode` class with `data` and `next` attributes. The `BinaryTreeNode` class has methods `insert` and `delete` for managing the tree. The code is written in Indonesian. The editor has a sidebar on the left showing project files. The bottom of the screen shows the Windows taskbar with various icons and the system clock. The taskbar includes the Start button, a search bar, and several application icons. The system clock shows the date as 28/05/2026 and the time as 13:36. The desktop background is a dark blue gradient with a large white 'X' shape.

Tugas 4 — add() dan traversePreOrder() di BinaryTreeArray (sudah ada di BinaryTreeArray00.java di atas) Method add() menambah data dengan cara increment idxLast lalu isi dataMahasiswa[idxLast]. Method traversePreOrder() mengunjungi node saat ini terlebih dahulu, kemudian rekursif ke kiri ($2*i+1$) lalu ke kanan ($2*i+2$).
Buatkan

[illegible]

Hasil Run Program setelah dimodifikasi:

[illegible]

LinkGitHub: <https://github.com/saumsaifulloh/AlgoritmaDanStrukturDataSem2/tree/main/ASD/PASD/JOBSHEET%2014>